

Teaching Computers to See: Building an Image Classifier with CNNs

Ahmad TAY, Faten CHAIEB, Hamza KCHOK, Asma GABIS

Objective

In this project, you will explore how Convolutional Neural Networks (CNNs) can be used to classify images. By the end, you will understand the basic architecture of CNNs and will build, train, and evaluate your own image classification model of cats and dogs.

Project Instructions

Step 1: Introduction to CNNs

- What are CNNs and how do they work?
- Explain key components: convolutional layers, pooling layers, activation functions, and fully connected layers.

Step 2: Dataset Exploration

- Load the dataset.
Remark: It is possible to use the `ImageDataGenerator` class from TensorFlow's `keras.preprocessing.image` module to load and preprocess images efficiently, especially when applying data augmentation. This is optional, but it can simplify many steps such as resizing, normalization, and augmentation.
- How many cat and dog images are there in the training and validation directories?
- Analyze class distribution and input shape.
- Visualize several images from the dataset.
- Split the training data into training and test sets (80%-20%).

Step 3: Build Your CNN Model

- Create a CNN using Keras and TensorFlow, including:
 - Four convolutional blocks.
 - Pooling layers (e.g., MaxPooling).
 - ReLU activation functions.
 - A fully connected (dense) output layer.
- Compile your model using an optimizer and a suitable loss function. Justify your choice.
- How many parameters does your model have in total? Calculate it **mathematically**.

Step 4: Train the Model

- Train your model for 5–15 epochs using a batch size of 100 samples.
- Track training and validation accuracy/loss over time.
- Look at the training and validation accuracy/loss graphs. Does your model perform much better on training data than on validation data (overfitting)? Or does it perform poorly on both (underfitting)? Explain what you see and why you think it happened.

Step 5: Evaluate the Model on the test set

- Display the confusion matrix.
- Calculate accuracy, precision, recall, and F1-score. What can you say about the model?

Step 6: Try to Improve the Model

Now we aim to improve our model. Apply each of the following techniques separately:

- **Data augmentation:** A technique used to artificially increase the size and diversity of a training dataset by applying random transformations like rotation, flipping, or zooming to the images. This helps the model generalize better and reduces overfitting.
Important: Data augmentation is used only on training and validation sets, not on the test set!
- **Dropout layers and batch normalization:** Add dropout to prevent overfitting and batch normalization to stabilize and accelerate training.

Bonus Challenge: Transfer Learning

Transfer learning is a technique where a model pre-trained on a large dataset (like ImageNet) is reused for a new, similar task. Instead of training from scratch, we fine-tune the existing model to save time and improve accuracy, especially when we have limited data. You can use any of the following models from TensorFlow Hub:

- MobileNetV2 (recommended), ResNet50, VGG16, etc.

Assessment Criteria

Criteria	Points / 100
Working CNN Model	30
Code Quality and Comments	20
Report Clarity and Insights	20
Evaluation and Metrics	20
Model Improvement Attempted	10
Creativity / Bonus Features	10

Note: Collaboration is encouraged, but plagiarism will result in disqualification. Make sure to cite all external sources used.

Deliverables

Each student or group must submit:

1. A Python notebook (.ipynb) containing all code, outputs, and comments.
2. A 10-minute presentation

“The best way to understand a system is to build it yourself.”